

# Reconstructing Training Data from Adapter-based Federated Large Language Models

Silong Chen  
National University of  
Defense Technology  
Changsha, China  
chensilong@nudt.edu.cn

Yuchuan Luo\*  
National University of  
Defense Technology  
Changsha, China  
luoyuchuan09@nudt.edu.cn

Guilin Deng  
National University of  
Defense Technology  
Changsha, China  
dengguilin@nudt.edu.cn

Yi Liu\*  
City University of Hong  
Kong  
Hong Kong, China  
yliu247-  
c@my.cityu.edu.hk

Ming Xu  
National University of  
Defense Technology  
Changsha, China  
xuming@nudt.edu.cn

Shaojing Fu  
National University of  
Defense Technology  
Changsha, China  
fushaojing@nudt.edu.cn

Xiaohua Jia  
City University of Hong  
Kong  
Hong Kong, China  
csjia@cityu.edu.hk

## Abstract

Adapter-based Federated Large Language Models (FedLLMs) are widely adopted to reduce the computational, storage, and communication overhead of full-parameter fine-tuning for web-scale applications while preserving user privacy. By freezing the backbone and training only compact low-rank adapters, these methods appear to limit gradient leakage and thwart existing Gradient Inversion Attacks (GIAs).

Contrary to this assumption, we show that low-rank adapters create new, exploitable leakage channels. We propose the Unordered-word-bag-based Text Reconstruction (UTR) attack, a novel GIA tailored to the unique structure of adapter-based FedLLMs. UTR overcomes three core challenges—low-dimensional gradients, frozen backbones, and combinatorially large reconstruction spaces—by: (i) inferring token presence from attention patterns in frozen layers, (ii) performing sentence-level inversion within the low-rank subspace of adapter gradients, and (iii) enforcing semantic coherence through constrained greedy decoding guided by language priors. Extensive experiments across diverse models (GPT2-Large, BERT, Qwen2.5-7B) and datasets (CoLA, SST-2, Rotten Tomatoes) demonstrate that UTR achieves near-perfect reconstruction accuracy (ROUGE-1/2 > 99), even with large batch sizes—settings where prior GIAs fail completely. Our results reveal a fundamental tension between parameter efficiency and privacy in FedLLMs, challenging the prevailing belief that lightweight adaptation inherently enhances security. Our code and data are available at link.

## CCS Concepts

• **Security and privacy**; • **Computing methodologies** → *Machine learning*;

\*Yuchuan Luo and Yi Liu are co-corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792182>

## Keywords

Federated Learning, Large Language Models, Gradient Inversion Attacks, Federated LLMs

## ACM Reference Format:

Silong Chen, Yuchuan Luo, Guilin Deng, Yi Liu, Ming Xu, Shaojing Fu, and Xiaohua Jia. 2026. Reconstructing Training Data from Adapter-based Federated Large Language Models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792182>

## 1 Introduction

In modern web applications, such as personalized web agents [5, 22], intelligent chatbots [20], and collaborative content generation platforms [36], there is an increasing demand for data and model collaboration across multiple users or devices to enhance performance and user experience. However, such collaboration often requires access to and processing of sensitive user private information, including personal messages, browsing histories, and transaction records, thereby raising serious privacy concerns [25]. Direct data sharing poses significant risks of data leakage or misuse, particularly under stringent data protection regulations such as the GDPR [35] and CCPA [3]. As a result, developing privacy-preserving techniques for collaborative learning has become an important research challenge in both academia and industry [9, 28].

Federated Large Language Model (FedLLM) [18, 40, 46] framework (e.g., Federatedscope-LLM [18]) has emerged as a promising solution to privacy concerns in collaborative learning by combining the powerful capabilities of LLMs with the privacy-preserving principles of FL. This paradigm enables decentralized model training without the need for centralized data collection [25]. In particular, multiple clients collaboratively train a shared global model by exchanging only local model updates (e.g., gradients or parameters) while keeping their raw data securely stored on local devices [23]. Given that LLMs typically contain millions or even billions of parameters, adapter-based FedLLM frameworks [1, 4, 24, 37] have become a mainstream approach within FedLLM systems to enhance training efficiency and reduce communication overhead. Adapters are

lightweight, parameter-efficient modules (e.g., Low-Rank Adaptation (LoRA) [15]) that can be seamlessly integrated into pre-trained LLMs, enabling fine-tuning for specific tasks with minimal computational and communication costs. By updating only a small subset of parameters, adapter-based FedLLMs achieve efficient model adaptation without compromising performance, making them particularly suitable for resource-constrained environments and expanding their applicability across diverse domains.

Despite these advantages, the security and privacy guarantees of adapter-based FedLLMs<sup>1</sup> may not be as robust as they appear. We argue this for two primary reasons. First, gradient inversion attacks [13, 29, 48, 49] are a well-established threat that can compromise data privacy by recovering sensitive training data from shared gradients. For instance, Zhu *et al.* [50] demonstrated the Deep Gradient Leakage (DGL) attack against traditional FL systems, revealing that adversaries can reconstruct private client data from exchanged gradients, posing one of the most severe privacy risks in collaborative learning. Second, similar to standard FL, adapter-based FedLLMs rely on the exchange of model parameters or gradients during training, inherently creating an attack surface for malicious participants or eavesdroppers. While such attacks are generally more challenging to execute in practical FedLLM settings, since existing methods primarily target full-parameter fine-tuning settings [1, 2, 10, 12, 32, 41] where gradients carry richer informational content, the risk is not entirely mitigated. Adapter-based FedLLMs update only a small subset of parameters, which can obscure certain data features and reduce the attack surface. Nevertheless, adaptive adversaries may still refine their techniques to exploit even sparse gradient information, indicating that privacy leakage remains a tangible and evolving threat in this paradigm.

In this paper, we disclose that adapter-based FedLLM frameworks remain vulnerable to serious privacy attacks and propose the Unordered-word-bag-based Text Reconstruction (UTR) attack, which is a new attack designed to recover sensitive training text. UTR pursues two primary objectives: (1) accurately reconstruct private textual training examples, and (2) evade common defensive mechanisms. We further demonstrate that UTR transfers effectively across a range of FedLLM configurations, including different adapter designs and underlying LLM backbones. Achieving such a powerful attack is non-trivial; below we summarize the three principal challenges we confronted:

- **C1: Lower Dimensionality of Updates.** Adapter-based frameworks update only a small fraction of parameters, drastically reducing gradient dimensionality and limiting raw information leakage. Consequently, traditional optimization-based gradient inversion attacks (e.g., DLG [50]), which depend on rich gradient signals, fail to reconstruct inputs accurately [10, 29].
- **C2: Frozen Backbone.** In adapter-based FedLLM, the backbone of the pre-trained LLM remains frozen, meaning its parameters are not updated during training [15]. Consequently, attackers can only access gradients from the adapter layers, while the key input-output mappings embedded in the attention and transformer layers of the backbone remain hidden. Even state-of-the-art methods such as DAGER [29], which can accurately reconstruct text

from attention layers in full-parameter tuning, suffer reduced inversion accuracy when such layers are inaccessible.

- **C3: Extensive Solution Space.** The solution space of possible reconstructions expands dramatically due to the large vector space induced by the embedding adapters. Direct traversal or beam search, as used in prior gradient inversion approaches [2, 10, 29], becomes computationally infeasible when the batch size is large (e.g.,  $\geq 32$ ).

To address C1, UTR abandons traditional gradient vector fitting methods (e.g., distance-based optimization) and instead leverages the frozen attention layers of the LLM to directly infer token presence from the vocabulary, determining whether a specific token was used during training. To address C2, we observe that adapter layers in frozen LLMs exhibit low-rank properties; UTR exploits the gradient-generated subspace of this structure to estimate the degree of sentence inversion, thereby enhancing reconstruction accuracy. To address C3, UTR incorporates grammatical, semantic, and contextual continuity constraints to drastically reduce the search space, applying a greedy search strategy for efficient and coherent text reconstruction. Moreover, we assess the degree of information leakage in FedLLMs, focusing on how much input data can potentially be inferred from a single gradient update. This provided insights into balancing privacy protection and performance trade-offs.

To validate the effectiveness of our attacks, we perform experiments using GPT-2 [33], BERT [16], and Qwen2.5-7B [45] on the popular datasets CoLA [38], SST [34], and Rotten Tomatoes [26] for batch sizes up to 128. We demonstrate that our attack can reconstruct large batch sizes with ROUGE-1/2  $> 99$  [19] in cases where the parameters are frozen. UTR significantly outperforms the state-of-the-art (SOTA) baselines (i.e., LAMP [2] and DAGER [29]), achieving improvements of at least 26 points for small batch sizes and 90 points for larger ones in terms of reconstruction quality (ROUGE-1/2). We also conduct defensive experiments by incorporating Differential Privacy (DP) [11] and Gradient Pruning (GP) [44] into the adapter updates. The results show that DP is a more effective defense method compared to GP, but it significantly undermines the model effectiveness and is not applicable to the real world. We summarize our contributions here:

- We demonstrate that even when most model parameters are frozen, adapter-based FedLLMs can still be reconstructed by an adversary with high probability.
- We propose a UTR attack to efficiently reconstruct specific discrete inputs in FedLLMs. Furthermore, we tailor the attack algorithm to different usage conditions.
- We show the extent of information leakage in FedLLMs, focusing on the conditions under which the model leaks more information based on the provided input. We find that the unidirectional attention mechanism, commonly used in generative LLMs, may pose a significant risk of information leakage.
- We experimentally show that our algorithm can not only reconstruct the input exactly but also effectively reconstruct complete batches of size  $b$  up to 128 in the adapter-based FedLLMs. The results indicate that our algorithm can also achieve high accuracy with ROUGE-1/2  $> 99$ .

<sup>1</sup>To avoid ambiguity in terminology, the terms “adapter-based FedLLMs” and “FedLLMs” are used interchangeably throughout the remainder of this paper.

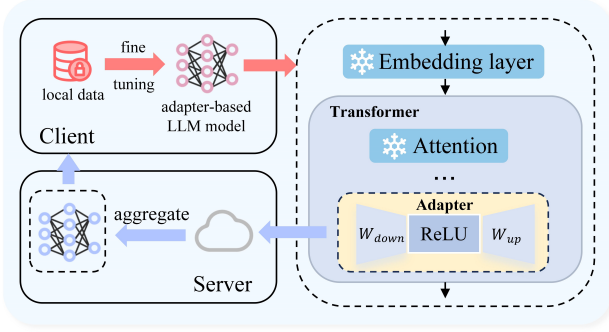


Figure 1: Standard architectures of adapter-based FedLLMs.

## 2 Related Work

### 2.1 Adapter-based FedLLMs

With the continuous development of LLMs, the parameter size of these models has increased significantly [4], amplifying the need for high-quality public data. FedLLMs naturally emerge as a solution to this challenge and have become a key direction in LLM development. However, due to the enormous parameter size of LLMs, directly applying FL for pre-training or full-parameter fine-tuning leads to substantial computational, storage, and resource overhead, which is impractical for most LLM users [42]. Therefore, reducing resource overhead is the primary challenge for achieving FedLLMs.

Current approaches include partial parameter fine-tuning [47], adapter-based fine-tuning [7, 8, 31], LoRA fine-tuning [15, 18, 43], prompt tuning [6, 12], and prefix tuning [21]. Among these, adapter-based fine-tuning, as illustrated in Figure 1, provides a practical trade-off between efficiency, performance, and privacy. By freezing the base model and training only lightweight adapter modules, this approach substantially reduces both communication and computation costs, making it well-suited for resource-constrained clients. It preserves the pretrained knowledge of the backbone while enabling effective personalization and multi-task adaptation across clients. Because only small adapter updates are shared, privacy risks and aggregation overhead are mitigated, while the modular design supports seamless integration of task- or domain-specific expertise [24]. Overall, adapter-based FedLLMs achieve better scalability and practicality than full fine-tuning [42], while typically offering stronger performance and flexibility than simpler approaches such as prompt or prefix tuning.

### 2.2 Gradient Inversion Attacks against FedLLMs

Gradient inversion attacks (GIAs) [48] aim to reconstruct training data by exploiting shared model gradients or parameters. These attacks pose severe privacy risks by exposing sensitive client information, motivating extensive research into privacy-preserving learning techniques. Early GIAs primarily targeted the image domain, where methods such as DGL [50] reconstruct images by minimizing the distance between reconstructed and original gradients. Building upon DGL, TAG [10] adapts this idea to Transformer-based models, improving textual reconstruction quality and reducing divergence from the true data. To overcome the inherent difficulty of

recovering discrete text from continuous gradients, LAMP [2] introduces an alternating continuous–discrete optimization strategy, achieving better reconstruction fidelity than TAG.

A conceptual shift was introduced by FILM [13], which departs from optimization-based approaches. FILM leverages gradient information from the embedding layer and the discrete nature of textual inputs to infer words directly. However, its performance is constrained by limited token sets and heuristic inference rules. DAGER [29] improves FILM by exploiting the one-to-one mapping between embedding outputs and discrete tokens, achieving nearly perfect reconstruction for small vocabularies—but at the cost of substantial computational inefficiency due to brute-force search.

In summary, existing GIAs largely focus on FedLLMs employing full-parameter fine-tuning, relying on gradients from embedding or Transformer layers for reconstruction. However, full-parameter tuning is computationally expensive and unrepresentative of mainstream adapter-based FedLLM paradigms, which typically freeze pre-trained parameters to reduce resource consumption. Consequently, the privacy risks of adapter-based FedLLMs remain under-explored, highlighting the need for systematic evaluation of their vulnerability to reconstruction attacks.

## 3 Background and Threat Model

### 3.1 Adapter-based FedLLMs

**Overview.** We begin by introducing the core components of the adapter-based FedLLM architecture, as illustrated in Fig. 1. The system comprises a server, multiple clients, the frozen parameters of a pre-trained LLM, and trainable adapter modules that can be inserted either within or between layers. The primary purpose of these adapters is to substantially reduce the number of trainable parameters, thereby lowering the computational, storage, and communication overhead. Based on their insertion locations, adapters can be categorized into two types: embedding adapters and layer adapters. Adapters placed after the embedding layer are referred to as embedding adapters, while those inserted within Transformer layers are called layer adapters. For example, the MAD-X [31] framework integrates adapters after the embedding layer, achieving strong cross-lingual and cross-task performance [17, 27, 31], and has since been incorporated into the Hugging Face library [32].

**Adapter.** As depicted in Fig. 1, each adapter is a lightweight, trainable module typically composed of three components: a downsampling layer, an activation function, and an upsampling layer [14, 30]. The downsampling layer projects high-dimensional hidden representations into a lower-dimensional bottleneck space, while the upsampling layer restores them to the original dimensionality. Although specific adapter implementations may vary, most share this fundamental design. A key hyperparameter in adapter configuration is the bottleneck dimension ( $d_{bottleneck}$ ), which is determined by the reduction factor ( $reduction_{factor}$ ). The reduction factor defines the ratio between the hidden layer dimension ( $d_{hidden}$ ) and the bottleneck dimension, formally expressed as:

$$reduction_{factor} = \frac{d_{hidden}}{d_{bottleneck}} \quad (1)$$

**Discussion.** This paper focuses exclusively on adapters following the aforementioned architecture, as they are the most widely

adopted in real-world deployments. This architecture is the basis for influential frameworks like MAD-X and is a standard option in libraries like FATE and Hugging Face *adapters*. On one hand, most existing studies employ this design, ensuring consistency and comparability with prior work. On the other hand, this architecture aligns with practical considerations in FedLLM scenarios, where computing resources are typically constrained, making parameter-efficient designs essential for feasible and scalable deployment. The attack would not directly apply to other PEFT approaches, which are out of scope.

### 3.2 Threat Model

**Adversary’s Goal.** Following the attack goals established in prior work, the adversary seeks to reconstruct as much of a victim client’s training data as possible using only the observable gradient information, thereby compromising client privacy. We consider the attack successful when the reconstructed output meaningfully matches the original training data—for example, when it attains high semantic similarity or otherwise preserves the sensitive content of the true examples.

**Adversary’s Identity.** The adversary could be any entity aiming to extract private information from the client. Examples include, but are not limited to, an honest-but-curious server, a rogue employee of a server company, a malicious third party controlling the server, a competing company, or a hacker. The method of obtaining the client’s gradients and the associated difficulty vary among these adversaries. For instance, an honest-but-curious server can efficiently perform the attack due to its role in model selection, parameter distribution, and gradient aggregation, which inherently provides the necessary knowledge and access. Similarly, rogue employees benefit from comparable advantages. In contrast, adversaries such as competing companies or hackers may need to steal gradient information actively through targeted attacks. For simplicity, we focus on the honest-but-curious server as the adversary in this study, representing the most common attack model in FedLLMs.

**Adversary’s Background Knowledge.** As noted above, we assume that the adversary has full knowledge of the pre-trained model’s architecture and parameters. In addition, since FedLLM training typically proceeds over multiple epochs, the adversary is assumed to observe the gradients shared by the victim client during each training round. However, the adversary cannot collude with any subset of clients and has no direct access to, or prior knowledge of, the client’s local training data.

## 4 Our Proposed UTR

### 4.1 Overview

To address the unique challenges of FedLLMs, i.e., the low dimensionality of updates, the frozen model backbone, and the extensive solution space, we propose UTR attack (see Fig. 2), a two-stage pipeline designed to systematically tackle these core issues. The key insight behind UTR is that coarse token recognition can be progressively refined into fine-grained sequence reconstruction by exploiting the intrinsic structural properties of adapter modules and frozen LLMs, enabling high-fidelity recovery of private training data. Next, we introduce the workflow of each stage as follows:

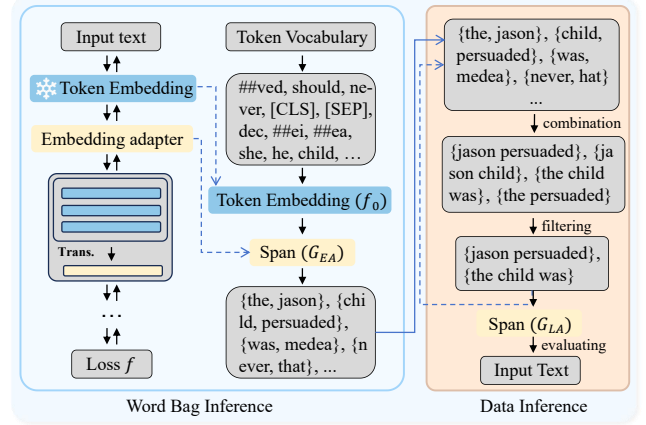


Figure 2: Overview of the proposed UTR attack.

**Stage 1: Word Bag Inference.** This initial stage targets the embedding adapter to infer an unordered set of vocabulary tokens present in the client’s private input. By leveraging the frozen embedding layer and the gradients from the embedding adapter, this stage bypasses traditional continuous optimization. It directly reduces the reconstruction search space, thereby addressing the challenge of low-dimensional updates (i.e., C1) (see §4.2).

**Stage 2: Data Inference.** This subsequent stage takes the output “word bag” from the first stage and assembles coherent text sequences. It employs a constrained search, guided by grammatical, semantic, and contextual filters to prune invalid token combinations. By using gradients from the layer adapter to perform membership inference, this stage navigates the extensive solution space (i.e., C3) and compensates for the inaccessibility of the frozen backbone (i.e., C2) (see §4.3).

**Discussion.** UTR deploys the two stages in a sequential pipeline, enabling near-exact reconstruction of client data even at large batch sizes and revealing a critical vulnerability. Importantly, UTR is model-agnostic, targeting the adapter modules as a universal attack surface to achieve high effectiveness across various LLM architectures and adapter configurations. Finally, we establish a Text Reconstruction Ability Assessment (see §4.4) to quantify the fundamental limits of this attack. This assessment defines the attack’s capability as the maximum number of words it can accurately reconstruct, which is fundamentally determined by the representational capacity of the low-rank space  $S$  derived from the adapter’s gradients.

### 4.2 “Word Bag” Inference Stage

**Key Idea.** The first stage identifies vocabulary tokens likely present in the client’s input. Exploiting gradients from the embedding adapter, UTR evaluates vocabulary items within the embedding space to determine token membership. This produces a “word bag”—a set of candidate tokens inferred to appear in the original input. Compared with traditional optimization-based methods, this approach avoids continuous vector fitting and leverages the discrete mapping between embeddings and tokens, substantially reducing the reconstruction search space. Specifically, in adapter-based FedLLMs, frozen embeddings can be treated as a deterministic function  $f_0$ , which maps input word indices to embedding vectors.

These embedding vectors are then passed to the embedding adapter as input. “Word bag” inference targets the embedding adapter by iterating over all words  $x$  in the model’s vocabulary, inputting them into the embedding layer, and checking whether the output  $f_0(x)$  matches the embedding adapter’s input data.

In this context, the “word bag” inference targeting an adapter essentially involves using the gradient information from the adapter to infer whether specific input data  $x$  belongs to the client’s input dataset. Here, we first analyze the properties of neurons in a generally fully connected (FC) layer. Let the input to the fully connected layer be a vector  $f(x)$ . For any neuron in this layer with weight  $\mathbf{W}$  and bias  $b$ , if the input vector  $f(x)$  activates the neuron, then the output  $y$  of the neuron can be expressed as:

$$y = \text{ReLU}(\mathbf{W} \cdot f(x) + b) = \begin{cases} \mathbf{W} \cdot f(x) + b, & \text{if } \mathbf{W} \cdot f(x) + b > 0. \\ 0, & \text{if } \mathbf{W} \cdot f(x) + b \leq 0 \end{cases} \quad (2)$$

According to the chain rule, the gradients of the loss  $L$  with respect to the weight  $\mathbf{W}$  and bias  $b$  are calculated as follows:

$$\begin{cases} \frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial b} \\ \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial \mathbf{W}} = \frac{\partial L}{\partial y} \cdot f(x) \end{cases} \quad (3)$$

Since  $\partial y / \partial b = 1$ , substituting this into Eq. (3) yields:

$$\frac{\partial f}{\partial \mathbf{W}} = \frac{\partial L}{\partial b} \cdot f(x). \quad (4)$$

From Eq. (4),  $f(x)$  can be calculated as follows:

$$f(x) = \frac{\frac{\partial f}{\partial \mathbf{W}}}{\frac{\partial L}{\partial b}} = \frac{\nabla \mathbf{W}}{\nabla b}, \quad (5)$$

where  $\nabla \mathbf{W}$  and  $\nabla b$  represent the gradients of the weights and biases, respectively. The ratio  $\nabla \mathbf{W} / \nabla b$  is defined as the Ratio of Weight to Bias Gradient (RWBG).

When multiple vectors are fed into the fully connected layer in a batch  $f(X)$ , it is possible that more than one vector simultaneously activates the neuron. In this case, the relationship between the gradient of the neuron’s weight and bias and the input data can be described as follows:

$$\frac{\nabla \mathbf{W}}{\nabla b} = \sum_i f(X)^i, \quad (6)$$

where  $f(X)^i$  is a vector in  $f(X)$  which can activate the neuron.

For the network architecture shown in Fig. 1, by calculating the RWBGs for all neurons in FC1, a linear space  $\mathbf{S}$  can be constructed based on the RWBGs, as follows:

$$\mathbf{S} = \begin{bmatrix} \frac{\nabla \mathbf{W}_{m,n}^{\text{up}_1}}{\nabla \mathbf{B}_m^{\text{up}_1}} \\ \dots \\ \frac{\nabla \mathbf{W}_{m,n}^{\text{up}_i}}{\nabla \mathbf{B}_m^{\text{up}_i}} \end{bmatrix}. \quad (7)$$

This linear space  $\mathbf{S}$  is essentially spanned by all the hidden embedding vectors fed into the linear layer. Since  $n > m$ , the space  $\mathbf{S}$  is a low-rank space, which results in a limited representational capacity. The hidden embedding vectors of LLMs are strongly correlated with the input data. Therefore, if there exists a hidden embedding vector in  $f(X)$  belongs to the space  $\mathbf{S}$ , it can be inferred that  $X$  is part of the current training batch of the client. The method to

determine whether a hidden embedding vector belongs to the space  $\mathbf{S}$  is to check if the hidden embedding vector can be expressed as a linear combination of the basis vectors of space  $\mathbf{S}$ . This is a straightforward computation in linear algebra.

### 4.3 Data Inference Stage

**Key Idea.** Following the Word Bag Inference stage, which produces an unordered set of candidate tokens  $W_b$ , the Data Inference stage aims to assemble these tokens into the original, coherent training text sequences. This stage systematically constructs and validates candidate sentences against the client’s private data by leveraging gradients from the layer adapter and applying a series of filtering constraints.

The core of this stage is a structured search process, as formalized in Algorithm 1 (Lines 12-34). The algorithm constructs candidate sentences by iteratively combining tokens from  $W_b$ . To manage the exponential complexity of this search space (C3), it employs a set of Boolean filter functions after each token extension: EICW() (Consecutive Identical Word Detection), CG() (Grammar Check) and CS() (Semantic Check). The specific implementation details are provided in Appendix B.

The final and critical step is membership inference, which verifies whether a candidate sentence matches the client’s true training data. This is achieved by feeding the candidate sequence through the FedLLM to obtain the corresponding hidden embeddings  $f_i(x)$  that serve as input to the first layer adapter. The core insight is that for the correct candidate, these hidden embeddings  $f_i(x)$  must lie within the low-rank subspace  $S_{LA}$  spanned by the basis vectors constructed from the client’s shared layer adapter gradients (as derived in Eq. (6)). The verification is performed by checking if  $f_i(x)$  can be expressed as a linear combination of the basis of  $S_{LA}$ , a direct computation in linear algebra.

**Impact of LLM Architecture.** The architecture of the underlying LLM influences the reconstruction outcome. For models with bidirectional attention mechanisms (e.g., BERT), the hidden embeddings are informed by the entire input context, enabling the reconstruction of the complete text in a single step. In contrast, for models with unidirectional attention (e.g., GPT), the hidden embedding for a token is only influenced by preceding tokens. This often results in the reconstruction of the text in a truncated or prefix-wise manner, requiring more candidate samples to assemble the full sequence.

Through this process of constrained candidate generation and gradient-based membership verification, the Data Inference stage successfully transforms an unordered bag of words into accurate reconstructions of the client’s private training texts. The algorithm description of the technical details of the above two stages is summarized in Algorithm 1 in the Appendix B.

### 4.4 Text Reconstruction Ability Assessment

The core mechanism of text reconstruction attacks involves inferring possible input words from the model’s vocabulary (“word bag”) and combining them into coherent text. The attack’s capability is therefore defined as the maximum number of words the algorithm can accurately reconstruct. When the client’s input exceeds this threshold, accurate reconstruction becomes infeasible.

We then analyze this threshold mathematically. Let a client's training batch contain  $b$  sequences comprising a total of  $n$  unique tokens. Each token  $t_i$  is mapped through the frozen embedding layer to a hidden representation vector  $v_i \in \mathbb{R}^{d_{\text{hidden}}}$ . The downsampling layer of the adapter, implemented as a fully connected layer, performs the following transformation, i.e.,  $Z = \text{ReLU}(WV + b)$ , where  $W \in \mathbb{R}^{d_{\text{bottleneck}} \times d_{\text{hidden}}}$  is the weight matrix,  $V \in \mathbb{R}^{d_{\text{hidden}} \times n}$  is the matrix of input hidden embeddings (each column corresponds to a token embedding  $v_i$ ),  $b \in \mathbb{R}^{d_{\text{bottleneck}}}$  is the bias vector, and  $Z \in \mathbb{R}^{d_{\text{bottleneck}} \times n}$  is the activated output.

**Prior knowledge.** As mentioned in §4.2, for the “word bag” inference to be successful, the hidden embedding vector  $v$  of a token must lie within the low-rank subspace  $S$  spanned by the client's actual input data. This is determined by analyzing the gradients. Following the Eqs. (4)–(7), for a single neuron  $j$  in the downsampling layer, the relationship between its weight gradient ( $\nabla W_j$ ), bias gradient ( $\nabla b_j$ ), and the input data is given by the RWBG:

$$RWBG_j = \nabla W_j / \nabla b_j = \sum_{l \in A_j} v_l, \quad (8)$$

where  $A_j$  is the set of indices of input vectors that activated neuron  $j$  (i.e.,  $W_j \cdot v_l + b_j > 0$ ). The subspace  $S$  is then constructed as the span of all these RWBG vectors across the  $d_{\text{bottleneck}}$  neurons:

$$S = \text{span}\{RWBG_1, RWBG_2, \dots, RWBG_{d_{\text{bottleneck}}}\} \quad (9)$$

Since each  $RWBG_j$  is a linear combination of the input vectors  $v_l$ , the subspace  $S$  is a subspace of  $\text{span}\{V\}$ , the column space of the input embedding matrix. Thus, the representational capacity of the attack is fundamentally constrained by the rank of the subspace  $S$ .

**THEOREM 4.1.** *Let  $V$  be a vector space over a field  $F$ . If  $S$  is a subspace of  $V$  spanned by a set of  $m$  vectors, then  $\dim(S) \leq m$ .*

**THEOREM 4.2. (Rank-Capacity.)** *The maximum number of linearly independent hidden embedding vectors  $v_i$  that can be perfectly represented in the subspace  $S$  is at most  $\text{rank}(S)$ . Further imore,  $\text{rank}(S) \leq \min(d_{\text{bottleneck}}, n, d_{\text{hidden}})$ .*

**PROOF.** For the specific proof details of Theorems 4.1 and 4.2, please refer to the Appendix A. Based on the theorem, we can get the Corollary below directly.  $\square$

**COROLLARY 4.3. (Maximum Recoverable Tokens.)** *The attack can, at most, uniquely identify and recover a number of tokens  $k_{\text{max}}$  such that  $k_{\text{max}} \leq \text{rank}(S) \leq d_{\text{bottleneck}}$ .*

## 5 Experiments

### 5.1 Experimental Setup

**Model and Dataset.** We will mainly carry out the main experiments on three datasets, i.e., CoLA [38], SST [34], and Rotten Tomatoes (RT) [26]. We evaluate the attack performance of the proposed UTR attack on three models: Bert-Base [16], GPT2-Large [33], and Qwen2.5-7B [45]. Detailed information about the models and datasets can be found in Appendix C.

**Training and Adapter Settings.** In the adapter-based FedLLMs, proper hyperparameter selection, i.e.,  $\text{reduction}_{\text{factor}}$ , is a notable factor that determines the proposed attack performance. We investigate the performance of the proposed framework with different

settings and try to find the best-performing threshold for it. In particular, we employ  $\text{reduction}_{\text{factor}} \in \{1, 2, 4, 8\}$  to adjust the best threshold of the proposed framework. The structure of adapters is set as Fig. 1, and we set the hyperparameter  $\text{reduction}_{\text{factor}} = 2$  in the main experiments of our paper. The others will be presented in the §5.4. We insert the adapters between the model's layers rather than inside. During the experimentation, we utilized only the first two Transformer layers of the model.

**Baselines.** We compare our attack with the following methods, including the optimization-based method (i.e., LAMP [2]) and the embedding-based method (i.e., DAGER [29]) designed for GIAs in FedLLMs. More details can be found in the Appendix D.

**Evaluation Metrics.** Following prior work, we evaluate the quality of text reconstruction using ROUGE-1/2 scores [19]. ROUGE-1 and ROUGE-2 are widely adopted metrics for assessing the quality of text summarization or machine-generated text by comparing it to human-written reference summaries. ROUGE-N is computed as follows:

$$\text{ROUGE} - N = \frac{\sum_i \text{Count}_i(\text{matched } n\text{-grams})}{\sum_i \text{Count}_i(n\text{-grams in Reference}_i)} \quad (10)$$

### 5.2 Attack Performance Evaluation

In this section, we aim to evaluate the effectiveness of the proposed UTR attack against adapter-based FedLLMs. We compare UTR's text reconstruction performance with baseline methods, i.e., LAMP and DAGER, across three benchmark datasets (CoLA, SST, and Rotten Tomatoes) and three representative model architectures (BERT, GPT2-large, and Qwen2.5-7B). An example of a UTR attack can be found in the Appendix E.

**Attack Performance under Different Data Volumes.** Specifically, we vary the batch size from 1 to 8 to evaluate performance under different data volumes. As shown in Table 1, UTR achieves nearly perfect text reconstruction, with ROUGE-1/2 scores approaching 100% across all models and datasets. In contrast, LAMP exhibits substantial performance degradation as batch size increases, while DAGER fails completely under the frozen backbone setting—highlighting UTR's superior robustness and adaptability. This performance gap arises because LAMP formulates reconstruction as an optimization problem, often resulting in unstable convergence, whereas UTR directly exploits gradient information for deterministic reconstruction. DAGER's failure, on the other hand, is attributed to its reliance on gradients from Transformer attention layers, which are typically frozen during fine-tuning. Note that as DAGER is not open-sourced, our implementation is based on its published description, which may lead to minor reproduction discrepancies.

**Attack Performance under Large Batch Size.** To evaluate scalability in practical large-batch scenarios, we evaluate the reconstruction capability of UTR on the GPT2-Large and Qwen2.5-7B models with substantially increased batch sizes. Specifically, we set the batch size to 16, 32, 64, and 128 on the CoLA, SST, and RT datasets. The results are shown in Table 2, where we discover that UTR maintains perfect or near-perfect reconstruction (ROUGE-1/2  $\approx 100$ ) for Qwen and on CoLA for GPT2-Large, even at a batch size of 128. However, we find that the attack on GPT2-Large fails on the SST and RT datasets for batches larger than 32, indicating that

**Table 1: Attack performance evaluation results. The R-1 and R-2 denote the ROUGE-1 and ROUGE-2 scores, respectively.**

| Batch | Method | Metrics | BERT       |            |            | GPT2         |              |            | Qwen       |            |            |
|-------|--------|---------|------------|------------|------------|--------------|--------------|------------|------------|------------|------------|
|       |        |         | CoLA       | SST        | RT         | CoLA         | SST          | RT         | CoLA       | SST        | RT         |
| B = 1 | Ours   | R-1     | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b>   | <b>100</b>   | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       |        | R-2     | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b>   | <b>99</b>    | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       | LAMP   | R-1     | 73.3       | 62.2       | 31.4       | 89.6         | 88.8         | 64.7       | /          | /          | /          |
|       |        | R-2     | 43.3       | 31.8       | 9.3        | 51.9         | 56.8         | 16.5       | /          | /          | /          |
|       | DAGER  | R-1     | 100        | 100        | 100        | 100          | 100          | 100        | 0          | 0          | 0          |
|       |        | R-2     | 100        | 100        | 100        | 100          | 100          | 100        | 0          | 0          | 0          |
| B = 2 | Ours   | R-1     | <b>100</b> | <b>100</b> | <b>100</b> | <b>99.83</b> | <b>100</b>   | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       |        | R-2     | <b>100</b> | <b>100</b> | <b>100</b> | <b>99.83</b> | <b>100</b>   | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       | LAMP   | R-1     | 26.8       | 21.4       | 11.2       | 77.8         | 82.4         | 46.4       | /          | /          | /          |
|       |        | R-2     | 11.0       | 9.2        | 0.9        | 31.5         | 45.7         | 7.6        | /          | /          | /          |
|       | DAGER  | R-1     | 100        | 99.3       | 98.1       | 100          | 100          | 100        | 0          | 0          | 0          |
|       |        | R-2     | 100        | 99.0       | 96.5       | 100          | 100          | 100        | 0          | 0          | 0          |
| B = 4 | Ours   | R-1     | <b>100</b> | <b>100</b> | <b>100</b> | <b>99.92</b> | <b>100</b>   | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       |        | R-2     | <b>100</b> | <b>100</b> | <b>100</b> | <b>99.88</b> | <b>99.25</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       | LAMP   | R-1     | 13.4       | 9.8        | 6.3        | 66.2         | 69.5         | 35.1       | /          | /          | /          |
|       |        | R-2     | 3.9        | 2.7        | 0.9        | 21.8         | 32.5         | 4.2        | /          | /          | /          |
|       | DAGER  | R-1     | 94.0       | 95.6       | 66.8       | 100          | 57.45        | 29.56      | 0          | 0          | 0          |
|       |        | R-2     | 89.9       | 93.0       | 50.1       | 100          | 52.63        | 21.65      | 0          | 0          | 0          |
| B = 8 | Ours   | R-1     | <b>100</b> | <b>100</b> | <b>100</b> | <b>99.92</b> | <b>100</b>   | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       |        | R-2     | <b>100</b> | <b>100</b> | <b>100</b> | <b>99.83</b> | <b>99.63</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> |
|       | LAMP   | R-1     | 8.9        | 8.1        | 6.8        | 52.9         | 56.9         | 27.3       | /          | /          | /          |
|       |        | R-2     | 1.9        | 0.7        | 0.3        | 13.1         | 19.1         | 2.0        | /          | /          | /          |
|       | DAGER  | R-1     | 67.8       | 74.1       | 37.1       | 88.18        | 18.51        | 20.37      | 0          | 0          | 0          |
|       |        | R-2     | 48.8       | 59.8       | 11.4       | 84.50        | 7.98         | 13.06      | 0          | 0          | 0          |

**Table 2: Main results on the GPT2-Large and Qwen-7B models with large batch sizes on various datasets.**

|       |      | B = 16 |       | B = 32 |       | B = 64 |       | B = 128 |       |
|-------|------|--------|-------|--------|-------|--------|-------|---------|-------|
|       |      | R-1    | R-2   | R-1    | R-2   | R-1    | R-2   | R-1     | R-2   |
| CoLA  | GPT2 | 100.0  | 100.0 | 100.0  | 100.0 | 100.0  | 100.0 | 100.0   | 100.0 |
|       | Qwen | 100.0  | 100.0 | 100.0  | 100.0 | 100.0  | 100.0 | 100.0   | 100.0 |
| SST-2 | GPT2 | 100.0  | 99.4  | 100.0  | 99.7  | 0.0    | 0.0   | 0.0     | 0.0   |
|       | Qwen | 100.0  | 100.0 | 100.0  | 100.0 | 100.0  | 100.0 | 100.0   | 100.0 |
| RT    | GPT2 | 100.0  | 100.0 | 100.0  | 100.0 | 0.0    | 0.0   | 0.0     | 0.0   |
|       | Qwen | 100.0  | 100.0 | 100.0  | 100.0 | 100.0  | 100.0 | 100.0   | 100.0 |

the unidirectional attention mechanism in GPT models may pose a greater challenge for reconstructing longer sequences from large batches.

### 5.3 Attack Performance Evaluation under Defenses

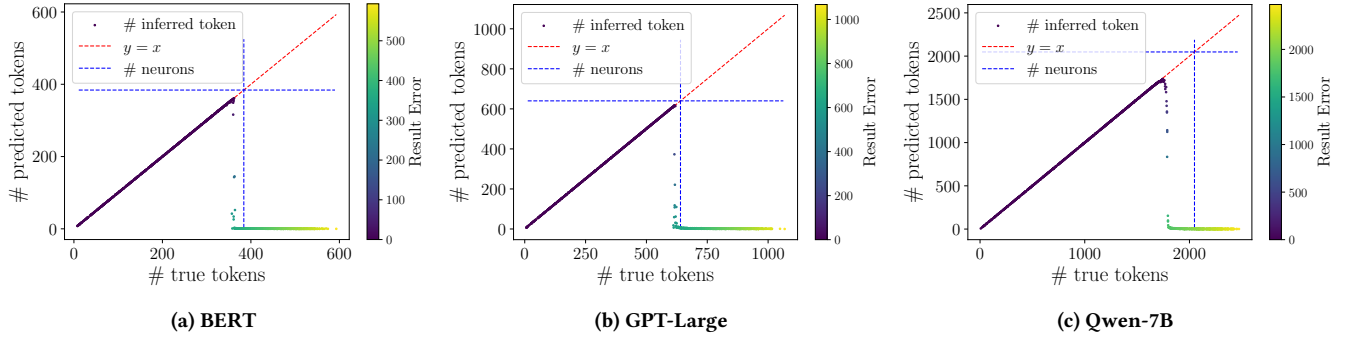
This section empirically evaluates the defensive efficacy of Differential Privacy (DP) [39] and Gradient Pruning (GP) [50]. We benchmark these defense methods against the UTR attack across model architectures, assessing their impact on both model utility and privacy protection. We implement a centralized DP framework where the server injects Gaussian noise into the aggregated adapter gradients from clients. The defense strength is primarily governed by the noise multiplier ( $\sigma$ ), for which we explore a range of values  $\sigma \in \{0.01, 1.5, 2.0, 3.0\}$  with a fixed clipping bound of 1.0. As a heuristic defense, GP discards gradient elements with absolute values below a specified threshold, thereby perturbing the gradient signal. We evaluate pruning rates ( $r$ )  $\in \{90\%, 95\%, 99.9\%\}$ , where  $r$  denotes the proportion of elements set to zero.

**Table 3: Effectiveness of defenses against our attack.**

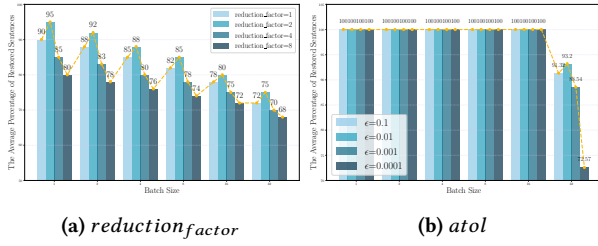
| Defense | Parameter       | BERT   |        | GPT2   |        | Qwen   |        |
|---------|-----------------|--------|--------|--------|--------|--------|--------|
|         |                 | R-1    | R-2    | R-1    | R-2    | R-1    | R-2    |
| DP      | $\sigma = 0.01$ | ~ 100% | ~ 100% | ~ 100% | ~ 100% | ~ 100% | ~ 100% |
|         | $\sigma = 1.5$  | ~ 15%  | ~ 15%  | ~ 13%  | ~ 17%  | ~ 0%   | ~ 0%   |
|         | $\sigma = 2.0$  | < 2%   | < 2%   | < 2%   | < 2%   | ~ 0%   | ~ 0%   |
|         | $\sigma = 3.0$  | ~ 0%   | ~ 0%   | ~ 0%   | ~ 0%   | ~ 0%   | ~ 0%   |
| GP      | $r = 90\%$      | ~ 65%  | ~ 65%  | ~ 65%  | ~ 65%  | ~ 60%  | ~ 60%  |
|         | $r = 99\%$      | ~ 25%  | ~ 25%  | ~ 25%  | ~ 25%  | ~ 26%  | ~ 25%  |
|         | $r = 99.9\%$    | < 5%   | < 5%   | < 5%   | < 5%   | < 5%   | < 5%   |

**Attack Performance under DP.** As shown in Table 3, the UTR attack achieved a 100% attack success rate at a noise level ( $\sigma = 0.01$ ) that does not destroy the effectiveness of the model. At  $\sigma = 1.0$ , the defense became nearly impenetrable, with success rates dropping below 2% and reconstructed text devolving into nonsensical word sequences (e.g., “Aesthetic volume conference bicycle”). A setting of  $\sigma = 1.5$  rendered the attack completely ineffective, achieving a 0% success rate. This demonstrates that DP provides a mathematically grounded and definitive defense against gradient inversion attacks, with its strength precisely controllable via the  $\sigma$  parameter. **However, we demonstrate in the Appendix F that such DP defense strength renders the model unusable.**

**Attack Performance under GP.** In contrast, GP offers fragile and unreliable protection. Under a mild pruning regime ( $r = 90\%$ ), the attack remained highly viable with a success rate of 65%, as the retained top gradients contained sufficient information for reconstruction. Aggressive pruning ( $r = 99\%$ ) reduced the success rate to approximately 25%, yet this still permitted significant leakage of key tokens and semantic concepts. It was only at an extreme, utility-destroying threshold ( $r = 99.9\%$ ) that the attack was effectively thwarted (success rate <5%). This fundamental limitation



**Figure 3: Relationship between the number of recovered tokens in the gradient information of the embedding adapter and the number of true tokens.**



**Figure 4: The performance across various  $reduction_{factor}$  and  $atol$  settings.**

underscores that GP’s protection is heuristic and brittle, failing to securely safeguard data without simultaneously negating the model’s learning capacity.

#### 5.4 Impact of the Adapter Settings

In this section, we further investigate the impact of the adapter’s bottleneck size and the span similarity threshold, controlled by the  $reduction_{factor} \in \{1, 2, 4, 8\}$ , and the  $\epsilon \in \{0.1, 0.01, 0.001, 0.0001\}$ , on the success of our reconstruction attacks.

**Impact of  $reduction_{factor}$ .** The results, illustrated in Fig. 4a, indicate a strong inverse correlation between the  $reduction_{factor}$  and attack performance. As the  $reduction_{factor}$  increases, the dimensionality of the adapter’s hidden representation decreases, creating a more severe information bottleneck. This bottleneck directly degrades the quality and completeness of the gradient information available for inversion. Consequently, while a larger  $reduction_{factor}$  offers improved parameter efficiency for federated tuning, it also inherently enhances privacy by attenuating the signal that can be exploited by gradient-based attacks.

**Impact of Span Similarity Threshold  $\epsilon$ .** The experimental results, illustrated in Fig. 4b reveal a critical trade-off between the interaction of batch size and solver tolerance (Atol). The key finding is that a higher tolerance value allows the attack to maintain a perfect or near-perfect success rate across a wide range of batch sizes, demonstrating that efficient large-batch processing can be achieved without performance loss. Conversely, when a very strict tolerance is enforced, the attack becomes highly fragile; its performance remains perfect for small batches but collapses dramatically as the batch size increases. This indicates that striving for high

numerical precision undermines the attack’s stability when scaling computations.

#### 5.5 Effectiveness Analysis of UTR Attack

This section evaluates the fundamental capability of adapter gradients to capture meaningful information, which directly dictates the upper bound of text reconstruction, as described in §4.4. To validate this, we conducted a large-scale experiment on the CoLA dataset, correlating the number of tokens reconstructed via the embedding adapter’s gradients against the true number of tokens in the original text. We varied the batch size from 1 to 256 to ensure a wide distribution of token counts, performing five rounds for a total of 1,280 trials.

The results, plotted in Fig. 3, show the relationship between the inferred and true token counts. Two key patterns emerge from the data. First, the effective information captured by the gradients is consistently slightly less than the total number of downsampling layer neurons. This aligns with our theoretical analysis in §4.4. Second, the amount of information effectively captured is positively correlated with the size of the adapter’s downsampling layer. A comparison of the subfigures in Fig. 3 confirms that the adapter in Qwen, with the most neurons, captures the most information. This pattern directly explains the superior attack performance of Qwen noted in prior sections.

### 6 Conclusion

This paper investigates the privacy risks of adapter-based FedLLMs under gradient inversion attacks (GIAs). We propose a unified framework for analyzing GIAs and uncover that adapter structures create new privacy vulnerabilities. To exploit these, we introduce UTR for data reconstruction by leveraging discrete LLM properties (e.g., positional embeddings, self-attention). Finally, we quantify information leakage from single gradient updates, offering insights into the privacy-performance trade-off. Our work advances understanding and defense of privacy in adapter-based FedLLMs.

#### Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62472431) and the Innovation Research Foundation of the National University of Defense Technology (Grant No. ZK25-17).

## References

- [1] Jiamu Bai, Daoyuan Chen, Bingchen Qian, Liuyi Yao, and Yaliang Li. 2024. Federated fine-tuning of large language models under heterogeneous tasks and client resources. In *Proc. of NeurIPS*.
- [2] Mislav Balunovic, Dimitar I. Dimitrov, Nikola Jovanovic, and Martin T. Vechev. 2022. LAMP: Extracting Text from Gradients with Language Model Priors. In *Proc. of NeurIPS*.
- [3] Rob Bonta. 2022. California consumer privacy act (CCPA). Retrieved from State of California Department of Justice: <https://oag.ca.gov/privacy/ccpa> (2022).
- [4] Dongqi Cai, Yaozong Wu, Shangguang Wang, Felix Xiaozhu Lin, and Mengwei Xu. 2023. Efficient federated learning for modern nlp. In *Proc. of Mobicom*.
- [5] Hongru Cai, Yongqi Li, Wenjie Wang, Fengbin Zhu, Xiaoyu Shen, Wenjie Li, and Tat-Seng Chua. 2025. Large language models empowered personalized web agents. In *Proc. of WWW*.
- [6] Tianshi Che, Ji Liu, Yang Zhou, Jiaxiang Ren, Jiwen Zhou, Victor S. Sheng, Huaiyu Dai, and Dejing Dou. 2023. Federated Learning of Large Language Models with Parameter-Efficient Prompt Tuning and Adaptive Optimization. In *Proc. of EMNLP*.
- [7] Haokun Chen, Yao Zhang, Denis Krompass, Jindong Gu, and Volker Tresp. 2024. FedDAT: An Approach for Foundation Model Finetuning in Multi-Modal Heterogeneous Federated Learning. In *Proc. of AAAI*.
- [8] Haokun Chen, Yao Zhang, Denis Krompass, Jindong Gu, and Volker Tresp. 2024. FedDAT: An Approach for Foundation Model Finetuning in Multi-Modal Heterogeneous Federated Learning. In *Proc. of AAAI*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.).
- [9] Shengchao Chen, Guodong Long, Tao Shen, and Jing Jiang. 2023. Prompt Federated Learning for Weather Forecasting: Toward Foundation Models on Meteorological Data. In *Proc. of IJCAI*.
- [10] Jieren Deng, Yijue Wang, Ji Li, Chenghong Wang, Chao Shang, Hang Liu, Sanguthevar Rajasekaran, and Caiwen Ding. 2021. TAG: Gradient Attack on Transformer-based Language Models. In *Proc. of Findings of EMNLP*.
- [11] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.
- [12] Tao Guo, Song Guo, and Junxiao Wang. 2023. Pfdprompt: Learning personalized prompt for vision-language models in federated learning. In *Proc. of WWW*.
- [13] Samyak Gupta, Yangsibo Huang, Zexuan Zhong, Tianyu Gao, Kai Li, and Danqi Chen. 2022. Recovering private text in federated learning of language models. In *Proc. of NeurIPS*.
- [14] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proc. of ICML*.
- [15] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. In *Proc. of ICLR*.
- [16] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proc. of NAACL-HLT*.
- [17] Mateusz Klimaszewski, Piotr Andruszkiewicz, and Alexandra Birch. 2025. No Train but Gain: Language Arithmetic for training-free Language Adapters enhancement. In *Proc. of ACL*.
- [18] Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proc. of KDD*.
- [19] Chin-Yew Lin. 2004. ROUGE: a Package for Automatic Evaluation of Summaries. In *Proc. of Workshop of ACL*.
- [20] Che Liu, Junfeng Jiang, Chao Xiong, Yi Yang, and Jieping Ye. 2020. Towards building an intelligent chatbot for customer service: Learning to respond at the appropriate time. In *Proc. of KDD*.
- [21] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-Tuning: Prompt Tuning Can Be Comparable to Fine-tuning Across Scales and Tasks. In *Proc. of ACL*.
- [22] Yi Liu, Cong Wang, and Xingliang Yuan. 2025. FedMobile: Enabling Knowledge Contribution-aware Multi-modal Federated Learning with Incomplete Modalities. In *Proc. of WWW*.
- [23] Yi Liu, Lei Xu, Xingliang Yuan, Cong Wang, and Bo Li. 2022. The right to be forgotten in federated learning: An efficient realization with rapid retraining. In *Proc. of INFOCOM*.
- [24] Guodong Long, Tao Shen, Jing Jiang, Michael Blumenstein, et al. 2024. Dual-personalizing adapter for federated foundation models. In *Proc. of NeurIPS*.
- [25] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. of AISTATS*.
- [26] Bo Pang and Lillian Lee. 2005. Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales. In *Proc. of ACL*.
- [27] Marinela Parovic, Alan Ansell, Ivan Vulic, and Anna Korhonen. [n. d.]. Cross-Lingual Transfer with Target Language-Ready Task Adapters. In *Proc. of Findings of ACL*.
- [28] Le Peng, Gaoxiang Luo, Sicheng Zhou, Jiandong Chen, Ziyue Xu, Ju Sun, and Rui Zhang. 2024. An in-depth evaluation of federated learning on biomedical natural language processing for information extraction. *NPJ Digit. Medicine* 7, 1 (2024).
- [29] Ivo Petrov, Dimitar Iliev Dimitrov, Maximilian Baader, Mark Niklas Mueller, and Martin Vechev. 2024. DAGER: Exact Gradient Inversion for Large Language Models. In *Proc. of NeurIPS*.
- [30] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proc. of EACL*.
- [31] Jonas Pfeiffer, Ivan Vulic, Iryna Gurevych, and Sebastian Ruder. 2020. MAD-X: An Adapter-Based Framework for Multi-Task Cross-Lingual Transfer. In *Proc. of EMNLP*.
- [32] Clifton Poth, Hannah Sterz, Indraneil Paul, Sukannya Purkayastha, Leon Engländer, Timo Imhof, Ivan Vulić, Sebastian Ruder, Iryna Gurevych, and Jonas Pfeiffer. 2023. Adapters: A Unified Library for Parameter-Efficient and Modular Transfer Learning. In *Proc. of EMNLP*.
- [33] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, Jeffrey Dean, and Sanjay Ghemawat. 2019. Language Models are Unsupervised Multitask Learners. In *Proc. of OSDI*.
- [34] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proc. of EMNLP*.
- [35] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A practical guide, 1st ed.*, Cham: Springer International Publishing 10, 3152676 (2017), 10–5555.
- [36] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, et al. 2024. Content-based collaborative generation for recommender systems. In *Proc. of CIKM*.
- [37] Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li. 2024. Flora: Federated fine-tuning large language models with heterogeneous low-rank adaptations. In *Proc. of NeurIPS*.
- [38] Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2019. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics* 7 (2019), 625–641.
- [39] Wenqi Wei, Ling Liu, Margaret Loper, Ka-Ho Chow, Mehmet Emre Gursay, Stacey Truex, and Yanzhao Wu. 2020. A framework for evaluating gradient leakage attacks in federated learning. *arXiv preprint arXiv:2004.10397* (2020).
- [40] Herbert Woitschläger, Alexander Erben, Shiqiang Wang, Ruben Mayer, and Hans-Arno Jacobsen. 2024. A Survey on Efficient Federated Learning Methods for Foundation Model Training. In *Proc. of IJCAI*.
- [41] Nan Wu, Xin Yuan, Shuo Wang, Hongsheng Hu, and Minhui Xue. 2024. Cardinality counting in "alcatraz": A privacy-aware federated learning approach. In *Proc. of WWW*.
- [42] Mengwei Xu, Dongqi Cai, Yaozong Wu, Xiang Li, and Shangguang Wang. 2024. FwdLLM: Efficient Federated Finetuning of Large Language Models with Perturbed Inferences. In *Proc. of USENIX ATC*.
- [43] Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng Zhang, and Qi Tian. 2024. QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models. In *Proc. of ICLR*.
- [44] Lulu Xue, Shengshan Hu, Ruizhi Zhao, Leo Yu Zhang, Shengqing Hu, Lichao Sun, and Dezhong Yao. 2024. Revisiting gradient pruning: A dual realization for defending against gradient attacks. In *Proc. of AAAI*.
- [45] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, and et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [46] Rui Ye, Wenhao Wang, Jingyi Chai, Dihan Li, Zexi Li, Yinda Xu, Yaxin Du, Yanfeng Wang, and Siheng Chen. 2024. Openfedllm: Training large language models on decentralized private data via federated learning. In *Proc. of KDD*.
- [47] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proc. of ACL*.
- [48] Rui Zhang, Song Guo, Junxiao Wang, Xin Xie, and Dacheng Tao. 2022. A Survey on Gradient Inversion: Attacks, Defenses and Future Directions. In *Proc. of IJCAI*.
- [49] Joshua C. Zhao, Atul Sharma, Ahmed Roushdy Elkordy, Yahya H. Ezzeldin, Salman Avestimehr, and Saurabh Bagchi. 2024. Loka: Large-scale Data Reconstruction Attack against Federated Learning through Model Manipulation. In *Proc. of SP*.
- [50] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. In *NeurIPS*.

**Algorithm 1** UTR Attack

---

**Require:** Embedding-Adapter gradients  $G_{EA}$ ,  
 Layer-Adapter gradients  $G_{LA}$ ,  
 vocabulary  $\mathcal{W}$ ,  
 projection thresholds  $\varepsilon_{EA}$  and  $\varepsilon_{LA}$ ,  
 maximum reconstruction length  $L_{\max}$ ,  
 and constrained beam width  $B$

**Ensure:** Reconstructed sentence candidates  $\mathcal{S}_{\text{cand}}$

**Preprocessing:**

- 1: Compute subspace bases:  $S_{EA} \leftarrow \text{span}(G_{EA})$ ,  $S_{LA} \leftarrow \text{span}(G_{LA})$
- 2: Define  $\text{in\_span}(v, S, \varepsilon)$  as the predicate
- 3:  $\|v - P_S(v)\|_2 < \varepsilon$ , where  $P_S(v)$  is the orthogonal projection of  $v$  onto  $S$

**Step 1: Infer Candidate Word Bag**

- 4: Initialize candidate word bag:  $\mathcal{W}_b \leftarrow \emptyset$
- 5: **for all**  $w \in \mathcal{W}$  **do**
- 6:    $v_w \leftarrow f_0(w)$  ▷ Word-level representation of  $w$
- 7:   **if**  $\text{in\_span}(v_w, S_{EA}, \varepsilon_{EA})$  **then**
- 8:      $\mathcal{W}_b \leftarrow \mathcal{W}_b \cup \{w\}$
- 9: **if**  $|\mathcal{W}_b| = 0$  **then**
- 10:   **return**  $\emptyset$  ▷ No recoverable words found

**Step 2: Sentence Reconstruction via Beam Search**

- 11: Initialize beam:  $\mathcal{B} \leftarrow \{(\text{seq} = [], \text{score} = 0)\}$
- 12: **for**  $l = 1$  to  $L_{\max}$  **do**
- 13:    $\mathcal{B}_{\text{new}} \leftarrow \emptyset$
- 14:   **for all**  $(\text{seq}, \text{score}) \in \mathcal{B}$  **do**
- 15:     **for all**  $w \in \mathcal{W}_b$  **do**
- 16:       **if**  $\text{EICW}(\text{seq}, w)$  **then** ▷ Exclude if  $w$  already in seq
- 17:       **continue**
- 18:        $\text{seq}' \leftarrow \text{seq} \oplus [w]$  ▷ Append  $w$  to sequence
- 19:       **if**  $\text{CG\_score}(\text{seq}')$  or  $\text{CS\_score}(\text{seq}')$  **then**
- 20:       **continue** ▷ Skip if grammatically implausible or semantically incoherent
- 21:        $v_{\text{seq}'} \leftarrow f_1(\text{seq}')$  ▷ Sequence-level representation
- 22:        $p_s \leftarrow \text{span\_similarity}(v_{\text{seq}'}, S_{LA})$
- 23:       **if**  $\text{in\_span}(v_{\text{seq}'}, S_{LA}, \varepsilon_{LA})$  **then**
- 24:        Add  $(\text{seq}', p_s)$  to  $\mathcal{B}_{\text{new}}$
- 25:   **if**  $\mathcal{B}_{\text{new}} = \emptyset$  **then**
- 26:     **break** ▷ No valid extensions; terminate early
- 27:   **Prune:** Keep top- $B$  sequences in  $\mathcal{B}_{\text{new}}$  with highest  $p_s$
- 28:    $\mathcal{B} \leftarrow \mathcal{B}_{\text{new}}$
- 29:   **if** convergence or early-stopping criterion is met **then**
- 30:     **break**

**Step 3: Return Final Candidates**

- 31: Sort  $\mathcal{B}$  by descending  $p_s$  to obtain  $\mathcal{S}_{\text{cand}}$
- 32: **return**  $\mathcal{S}_{\text{cand}}$

---

**A Proofs**

The representational capacity of the attack is fundamentally constrained by the rank of the subspace  $S$ . We then prove Theorems 4.1 and 4.2 as follows:

**PROOF.** Let  $W = \mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_m$  be a spanning set for  $S$ . If  $W$  is linearly independent, then  $\dim(S) = m$ . If  $W$  is linearly dependent,

then there exists a proper subset of  $W$  that forms a basis for  $S$ , and thus,  $\dim(S) < m$ . Therefore, in all cases,  $\dim(S) \leq m$ .  $\square$

**PROOF.** We prove the theorem by establishing the bounds on the rank of  $S$  through the following arguments:

**Step 1:** By definition, the subspace  $S$  is spanned by the  $d_{\text{bottleneck}}$  vectors  $RWBG_j$ . By Theorem 4.1, the dimension of a subspace cannot exceed the number of vectors in its spanning set, therefore we have:

$$\text{rank}(S) \leq d_{\text{bottleneck}}. \quad (11)$$

**Step 2:** Each  $RWBG_j$  is a linear combination of the  $n$  input hidden embedding vectors  $v_i^n$ . Therefore,  $S$  is a subspace of the span of  $v_i^n$ , which has dimension at most  $\min(n, d_{\text{hidden}})$ . Thus:

$$\text{rank}(S) \leq \min(n, d_{\text{hidden}}) \quad (12)$$

**Step 3:** Combining the bounds from Step 1 and Step 2, the most restrictive bound is the minimum of these three values. In practical FedLLM scenarios,  $d_{\text{bottleneck}}$  is typically much smaller than both  $n$  and  $d_{\text{hidden}}$  (e.g.,  $d_{\text{bottleneck}} = 512$ ,  $d_{\text{hidden}} = 4096$ ,  $n$  can be in the thousands for a large batch). Therefore, the bottleneck dimension is the effective upper limit:

$$\text{rank}(S) \leq \min(d_{\text{bottleneck}}, n, d_{\text{hidden}}) \quad (13)$$

Finally, the maximum number of linearly independent vectors that can be perfectly represented in  $S$  is precisely  $\text{rank}(S)$ , by the definition of dimension. This completes the proof.  $\square$

We analyze UTR attacks within an ideal setting to delineate the theoretical limits of their efficacy. By characterizing this upper bound, we provide the necessary analytical groundwork to inform the design of provably resilient countermeasures.

**B Algorithm**

The pseudo code of the detailed execution steps of the UTR attack is shown in the following algorithm. The algorithm constructs candidate sentences by iteratively combining tokens from  $\mathcal{W}_b$ . To manage the exponential complexity of this search space (C3), it employs a set of Boolean filter functions after each token extension: EICW() (Consecutive Identical Word Detection): Prunes candidates containing repeated consecutive tokens, which are uncommon in valid text. CG() (Grammar Check): Filters out sequences that exhibit obvious syntactic errors. It is evaluated using *language\_tool\_python* with over 50 syntactic rules. CS() (Semantic Check): Discards combinations that lack basic semantic coherence. We measure it via cosine similarity  $> 0.2$  in SBERT embedding space. These filters collectively guide the search towards linguistically plausible sequences, drastically reducing the number of candidates that require computationally expensive verification.

**C Models and Datasets**

Table 4 provides key information about these models, including whether their embedding layers incorporate positional encoding mechanisms, whether they utilize unidirectional or bidirectional attention mechanisms, and their hidden dimension sizes. These features are identified as critical because they influence the implementation of the attack and impact the effectiveness of the attack. A detailed analysis of these factors will be conducted during the experimental process.

**Table 5: Basic information of the dataset.**

| Dataset | Min text len | Max text len | Ave. text len |
|---------|--------------|--------------|---------------|
| CoLA    | 2            | 42           | 7.7           |
| SST     | 2            | 52           | 19.14         |
| RT      | 1            | 59           | 20.99         |

**Table 4: Basic information of models. PE means whether the model has position embedding.**

| Model      | PE | Self/Bidirect | Hidden Embedding |
|------------|----|---------------|------------------|
| BERT-Base  | ✓  | Bidirect      | 786              |
| GPT2-Large | ✓  | Self          | 1280             |
| Qwen2.5-7B | ×  | Bidirect      | 4096             |

This work leverages three benchmark datasets, i.e., the Corpus of Linguistic Acceptability (CoLA), the Stanford Sentiment Treebank (SST-2), and the Rotten Tomatoes (RT) movie review dataset, each probing a distinct facet of this capability. The numerical information of the above dataset can be found in Table 5.

**CoLA Dataset.** CoLA is a specialized dataset designed to evaluate a model’s fundamental grasp of English syntax and grammatical structure. Unlike tasks focused on meaning or sentiment, CoLA frames language understanding as a binary classification problem of grammaticality judgment, where a model must determine whether a given sentence is linguistically acceptable or unacceptable. The sentences are drawn from expert linguistics publications and annotated by trained linguists, ensuring high-quality labels for complex grammatical phenomena. This includes testing for subtle violations in subject-verb agreement, verb argument structure, and anomalous word sequences. As a core component of the GLUE benchmark, CoLA provides a crucial test of a model’s syntactic competence, probing its ability to internalize the formal rules of the language rather than just statistical patterns in the data.

**SST Dataset.** SST-2 offers a comprehensive benchmark for fine-grained sentiment analysis by moving beyond simple document-level classification. Derived from movie reviews, this dataset is unique in that it provides sentiment labels not only for entire sentences but for every single phrase and constituent within their parse trees. This rich, granular annotation allows models to be trained and evaluated on understanding how sentiment is compositionally built from smaller units, capturing the nuanced impact of negations, contrasts, and intensifiers. The task is a binary classification of sentiment into positive or negative categories. SST-2’s primary challenge and significance lie in pushing models to develop a more profound, structured understanding of sentiment composition, making it a standard for assessing nuanced semantic understanding.

**RT Dataset.** RT movie review dataset serves as a classic and widely-adopted benchmark for binary sentiment classification at the document or snippet level. Sourced from the critical review aggregator website, it consists of raw text snippets from professional movie reviews, each labeled with a broad positive or negative sentiment. In contrast to the phrase-level detail of SST-2, the RT dataset typically presents a more direct and practical sentiment analysis task: classifying the overall polarity of a given text segment.

Its significance stems from its realism and simplicity, providing a robust testbed for a model’s performance on a straightforward, real-world opinion mining task, and it is often used in direct comparison with other sentiment datasets to evaluate generalizability and robustness.

## D Baselines

We compare our attack with the following methods, including the optimization-based method (i.e., LAMP [2]) and the embedding-based method (i.e., DAGER [29]) designed for GIAs in FedLLMs.

**LAMP.** LAMP [2] is a novel and highly effective reconstruction attack designed to extract private, client-held text data from the gradient updates exchanged during the federated learning of transformer-based language models. The core innovation of LAMP lies in its two-pronged attack strategy, which alternates between continuous and discrete optimization. The continuous phase refines a set of dummy token embeddings by minimizing the difference between their resulting gradients and the target client’s true gradients. Crucially, to overcome local optima and guide the reconstruction towards coherent text, LAMP introduces a discrete optimization phase that uses an auxiliary language model, such as GPT-2, as a prior. This phase generates candidate sequences through textual transformations and selects the one that best balances gradient-matching loss with low linguistic perplexity. This synergistic approach enables LAMP to significantly outperform prior attacks, successfully reconstructing longer, more accurate text sequences, including in challenging scenarios with batch sizes larger than one and against defended models, thereby exposing a greater vulnerability to privacy leakage in federated learning systems than previously known.

**DAGER.** DAGER [29] is a groundbreaking gradient inversion attack that, for the first time, achieves the exact reconstruction of entire batches of private text input to large language models (LLMs) from their gradients. Departing from the approximate, optimization-based approaches of prior methods, DAGER leverages two key properties of transformer models: the low-rank structure of self-attention layer gradients and the discrete nature of tokenized text. Its core mechanism involves a highly efficient “span check” that verifies whether a candidate token’s embedding lies within the column space of the observed gradient matrices. This allows DAGER to first recover the set of all tokens present in a client’s batch and then, through a greedy algorithm for decoders or a heuristic search for encoders, reassemble them into the original sequences with perfect accuracy. This approach enables DAGER to scale to significantly larger batch sizes (up to 128) and longer sequences than previously possible, achieving near-perfect ROUGE scores and dramatically outperforming prior attacks in both reconstruction quality and computational speed.

**Discussion.** During experiments, we employed only the first two Transformer layers for two reasons. First, UTR requires gradient data from the embedding and first-layer adapters to evaluate privacy leakage, and these are fully captured within the initial layers. Second, due to computational constraints—specifically, a single NVIDIA RTX 4090 GPU (24 GB VRAM) on a Windows system—running the full model was infeasible. Thus, restricting to two layers provided an effective and practical balance between experimental validity and resource limitations.

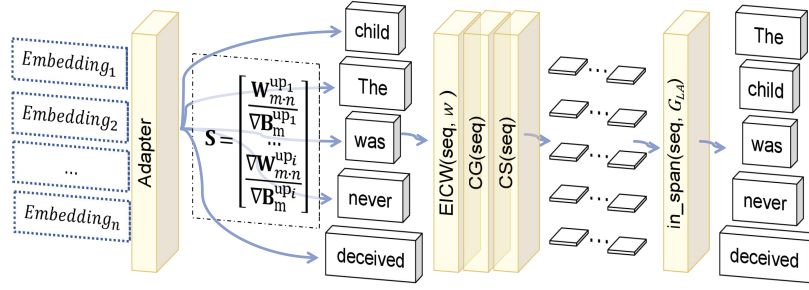


Figure 5: The showcase of the UTR attack.

Table 6: An example of UTR on CoLA with batch size 2.

|             |                                                                                           |                                                                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Origin text | The child was never deceived.<br>Jason persuaded Medea that she should desert her family. |                                                                                                                                                                         |
| BERT        | word bag                                                                                  | '##ved', 'should', 'never', 'desert', 'dec', 'med', 'the', 'was ', 'child', 'that', 'her', 'she', '[CLS]', '[SEP]', 'family', 'jason', '.', 'persuaded', '##ei', '##ea' |
|             | Recon. text                                                                               | the child was never deceived.<br>jason persuaded medea that she should desert her family.                                                                               |
| GPT         | word bag                                                                                  | ('.', 'was ', 'that', 'The', 'her', 'she', 'should', 'child', 'never', 'family', 'Med', 'desert', 'ea', 'persuaded', 'Jason', 'deceived', '< endoftext >')              |
|             | Recon. text                                                                               | the child was never deceived.<br>Jason persuaded Medea that she should desert her family.                                                                               |
| Qwen        | word bag                                                                                  | ('.', ' that', ' was ', 'The', ' her', ' should', ' she', ' child', ' never', 'ea', ' desert', 'Jason', ' persuaded', ' deceived')                                      |
|             | Recon. text                                                                               | The child was never deceived.<br>Jason persuaded Medea that she should desert her family.                                                                               |

Table 8: Computational resource overhead for data reconstruction across methods and tasks.

| Task      | Method | B   | GPU Time (s) |
|-----------|--------|-----|--------------|
| RT-GPT2   | UTR    | 128 | 6,125        |
| RT-GPT2   | FILM   | 128 | ~26,000      |
| CoLA-GPT2 | UTR    | 64  | 900          |
| CoLA-GPT2 | FILM   | 64  | ~54,000      |

Table 7: Model utility evaluation under DP.

| Defense | Parameter         | BERT   |          |
|---------|-------------------|--------|----------|
|         |                   | Loss   | Accuracy |
| DP      | $\sigma = 0.0001$ | ~ 0.42 | ~ 81.2%  |
|         | $\sigma = 10.0$   | ~ 0.61 | ~ 69.2%  |

## E Attack Example of UTR

**Sample Reconstruction.** We show sample sequence reconstructions from UTR on CoLA with  $B = 2$  in Table 6. We can observe that our reconstruction is coherent, which can successfully reconstruct the accurate original text and Fig. 5 illustrates the showcase of our UTR.

## F Additional Experimental Results

**Model Utility Evaluation under DP.** As illustrated in Table 7, when strong Differential Privacy (DP) guarantees are enforced, the model’s utility drops to 69.20%. This performance level remains substantially below what is considered sufficient for real-world deployment (80%-82%).

### Attack Performance Evaluation under Structural Defense.

In response to the suggestion regarding structural defenses, we note that while such architectural modifications are generally avoided to prevent client-side heterogeneity, they offer valuable insights in an experimental context. Our supplementary tests reveal that architectural hardening provides measurable benefits: substituting ReLU with GELU lowered reconstruction by  $\approx 15\%$ , while utilizing deeper adapters (depth=3) reduced leakage by  $\approx 10\%$  ( $B < 64$ ) and  $\approx 80\%$  ( $B \geq 64$ ). These results align with and support our primary defense analysis.

**Attack Efforts Evaluation.** We further evaluated the computational efficiency of UTR on open-source benchmarks. As summarized in Table 8, UTR achieves significantly lower computational overhead compared to state-of-the-art methods. For instance, on the RT dataset ( $B = 128$ ), UTR requires only 6,125 GPU-seconds ( $\sim 1.7$  GPU-hours) to reconstruct *all* batches, which is over  $5\times$  more efficient than FILM. Moreover, on the CoLA-GPT2 task ( $B = 64$ ), UTR completes reconstruction in approximately 15 minutes, yielding a speedup of more than  $60\times$  relative to FILM.